Reviewer Pack — Minimal Rank of Quandle Representations vs Resolution Proof …

Entry 02ea1d2603cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 23:37:49 UTC

Minimal Rank of Quandle Representations vs Resolution Proof Length for Tseitin Formulas

Entry ID: `02ea1d2603cb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 23:37:49 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For any Tseitin formula F on an expander graph G, there exists a quandle representation Q of the vertex set V(G) such that the resolution proof length for F is at least 2^Ω(rank(Q)).', 'The rank of Q is poly-time computable and is O(1) for all graphs that are not expanders.', "If F has a resolution refutation of length L, then there exists a quandle representation Q' of V(G) such that the rank of Q' is at most 2^L."]

Rationale (proposer's reasoning):

> ['Quandle theory, which studies algebraic structures related to symmetry and group actions, provides a framework for studying invariants of graphs. By leveraging this theory, we may uncover new graph properties that are relevant to the complexity of Tseitin formulas.', 'The connection between quandles and resolution proof length suggests that the structure of a graph can be used to predict the difficulty of resolving Tseitin formulas, potentially leading to new lower bounds.', 'This conjecture builds on previous work in graph theory and complexity theory by introducing a novel mathematical tool, the quandle representation, which has not been previously applied to resolution proof complexity.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b6a37a972e51a93a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated Tseitin formulas on expander graphs with n ≤ 40 vertices, the ratio of the average resolution proof length to the average rank of quandle representations is at least 2^Ω(rank(Q)) and no quandle representation has a

rank less than 2^(resolution proof length - 1). The conjecture is falsified if this relationship does not hold for any generated formula.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quandle Theory" AND "Resolution Proof Complexity" AND Tseitin Formula" - "Minimal Rank Quandle Representation" AND expander graph" - "resolution proof length" IN "Quandle Theory" OR "Complexity Theory: Resolution Proof Complexity"

Top relevant hits considered: - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2509.13291v1] Towards an Embodied Composition Framework for Organizing Immersive Computational Notebooks

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_expander_graph(n):
        G = {}
        for i in range(n):
            G[i] = set()
        for _ in range(2 * n - 1):
            u, v = random.sample(range(n), 2)
            G[u].add(v)
            G[v].add(u)
        return G

    def generate_tseitin_formula(G):
        clauses = []
        literals = {}
        for i in G:
```

```
            literals[i] = (random.choice(['A', 'B']), random.choice([True, False]))
            if literals[i][1]:
                clauses.append([(literals[i][0], i)])
            else:
                clauses.append([(-literals[i][0], i)])
        for u in G:
            for v in G[u]:
                a = (random.choice(['A', 'B']), random.choice([True, False]))
                b = (random.choice(['A', 'B']), random.choice([True, False]))
                c = (random.choice(['A', 'B']), random.choice([True, False]))
                literals[(u, v)] = (a[0], b[0], c[0])
                clauses.append([(literals[(u, v)][0], u), (-literals[(u, v)][1], v)])
                clauses.append([(-literals[(u, v)][0], v), (literals[(u, v)][1], u)])
                clauses.append([(-literals[(u, v)][2], u), (-literals[(u, v)][2], v)])
        return clauses

    def compute_quandle_representation(G):
        quandles = {}
        for i in G:
            quandles[i] = set()
        for u in G:
            for v in G[u]:
                if (u, v) not in quandles and (v, u) not in quandles:
                    quandles[(u, v)] = random.choice(['A', 'B'])
                    quandles[(v, u)] = quandles[(u, v)]
        return quandles

    def compute_rank(quandle):
        n = len(quandle)
        matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if (i, j) in quandle:
                    matrix[i][j] = 1
                    matrix[j][i] = 1

        def gaussian_elimination(A):
            m, n = len(A), len(A[0])
            rank = 0
            for i in range(n):
                pivot_row = -1
                for j in range(rank, m):
                    if A[j][i]:
                        pivot_row = j
                        break
                if pivot_row == -1:
                    continue
                A[pivot_row], A[rank] = A[rank], A[pivot_row]
                rank += 1
                for j in range(n):
                    if j != i and A[pivot_row][j]:
                        factor = Fraction(A[j][i], A[pivot_row][i])
                        for k in range(n):
                            A[j][k] -= factor * A[pivot_row][k]
            return rank
```

```
        return gaussian_elimination(matrix)

    def compute_resolution_proof_length(clauses):
        assignment = {}

        def dpll(clauses, assignment, Q, V):
            if not clauses:
                return True
            unit_clause = next((c for c in clauses if len(c) == 1), None)
            if unit_clause:
                literal = unit_clause[0]
                if literal[0] in assignment and assignment[literal[0]] != literal[1]:
                    return False
                assignment[literal[0]] = literal[1]
                clauses = [c for c in clauses if not any(v in assignment or (assignment[v] == 'T'
            pure_literal = next((v for v in V if sum(1 for c in clauses if v in c) - sum(1 for c i
            if pure_literal:
                assignment[pure_literal] = True
                clauses = [c for c in clauses if not any(v in assignment or (assignment[v] == 'T'
            return dpll(clauses, assignment, Q, V)

        return len(clauses) if not dpll(clauses, assignment, {}, range(len(clauses))) else 1

    n = random.choice([5, 10, 15, 20, 30, 40])
    G = generate_expander_graph(n)
    F = generate_tseitin_formula(G)

    quandle_representations = [compute_quandle_representation(G) for _ in range(30)]
    ranks = [compute_rank(qr) for qr in quandle_representations]
    proof_lengths = [compute_resolution_proof_length(F) for _ in range(30)]

    mean_rank = sum(ranks) / len(ranks)
    mean_proof_length = sum(proof_lengths) / len(proof_lengths)
    support_fraction = sum(1 for rank, length in zip(ranks, proof_lengths) if length >= 2 ** math.

    conjecture_holds = support_fraction >= 0.8
    counterexample = "" if conjecture_holds else "support_fraction < 0.8"

    return {
        "metric_name": "Rank vs DPLL Length",
        "metric_value": mean_rank,
        "instances_tested": len(ranks),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **result}}")
        results.append(result)
```

```
    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    mean_length = sum(r["instances_tested"] * r["metric_value"] for r in results) / sum(r["instanc
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='support_fraction < 0.8' first_failing_seed={firs
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5ff98983.py", line 143, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5ff98983.py", line 116, in run_trial
    F = generate_tseitin_formula(G)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5ff98983.py", line 39, in generate_tsei
    clauses.append([(-literals[i][0], i)])
                     ^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete its execution and provide a result. Therefore, the conjecture cannot be supported or falsified based on this test. | next: Investigate the cause of the crash in the test code and attempt to run the test again to obtain valid results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13176
2	propose	ollama_remote	glm4:latest	0	0	13616
3	preregistration	ollama_remote	glm4:latest	0	0	6566
4	novelty	ollama_remote	glm4:latest	0	0	5189
5	novelty	ollama_remote	glm4:latest	0	0	7100
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41705
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10299
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12509
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19223
10	judge	ollama_remote	glm4:latest	0	0	11004

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 140388 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/02ea1d2603cb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/02ea1d2603cb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/02ea1d2603cb.tar.gz` (if generated)